Claude Code

By

Anand Kulkarni

anand.kulkarni1@zensar.com

Table of Content

Module	Topic
Module 1:	Introduction to Anthropic Claude
Module 2:	Claude products with pricing
Module 3:	Claude Models
Module 4:	Claude code installation
Module 4:	Claude slash commands
Module 5:	How Claude code works?
Module 6:	Claude.md file
Module 7:	Settings.json file
Module 8:	Claude modes

Introduction to Anthropic Claude

- Anthropic Claude is a family of advanced AI models developed by Anthropic, an AI research and safety company.
- It is designed to be helpful, reliable, and safe, Claude can understand and generate natural language, answer questions, summarize documents, write and debug code, analyze data, and assist with research and creative tasks.
- It is particularly useful for working with long documents, complex reasoning, business communication, and software development.
- It is available through web-based interfaces, APIs and integrations with various business tools. Anthropic Claude serves individuals, developers, and organizations seeking to improve productivity, automate workflows and make better decisions using GenAI.

Claude products with pricing

SR. NO.	PRODUCT NAME	USAGE
1	Claude Code	Claude code is an AI coding tool that helps developers understand codebases, modify files, debug issues, and automate software development tasks.
2	Claude Cowork	Claude Cowork is an agent-style tool for delegating multi-step knowledge-work tasks, such as organizing files and working across documents, preparing presentation etc.
3	Claude Design	Claude Design helps users create and refine visual designs, prototypes, and other design materials using natural-language instructions.
4	Claude for Microsoft 365	Claude for Microsoft 365 integrates Claude into supported Microsoft productivity workflows to help users work with documents, email, and business information.
5	Claude Science	Claude Science supports research-oriented work, including scientific analysis and complex information exploration. Availability depends on the plan and feature rollout.

Claude pricing for individuals



Free

Try Claude

$0

Free for everyone

Try Claude

- ✓ Chat on web, desktop, and mobile
- ✓ Search the web, create files, and run code
- ✓ Memory across conversations
- ✓ Connect your apps and tools
- ✓ Create Artifacts
- ✓ Claude Design, Slides, Docs



Pro Claude Code included

For everyday work

$17

Per month with annual subscription discount ($200 billed up front). $20 if billed monthly.

Try Claude

Everything in Free, plus:

- ✓ More usage*
- ✓ Hand off and schedule tasks
- ✓ Claude Dashboards (beta)
- ✓ Claude Science
- ✓ Projects
- ✓ More Claude models
- ✓ Claude in Chrome and Microsoft 365



Max Claude Code included

For people who work with Claude all day

From $100

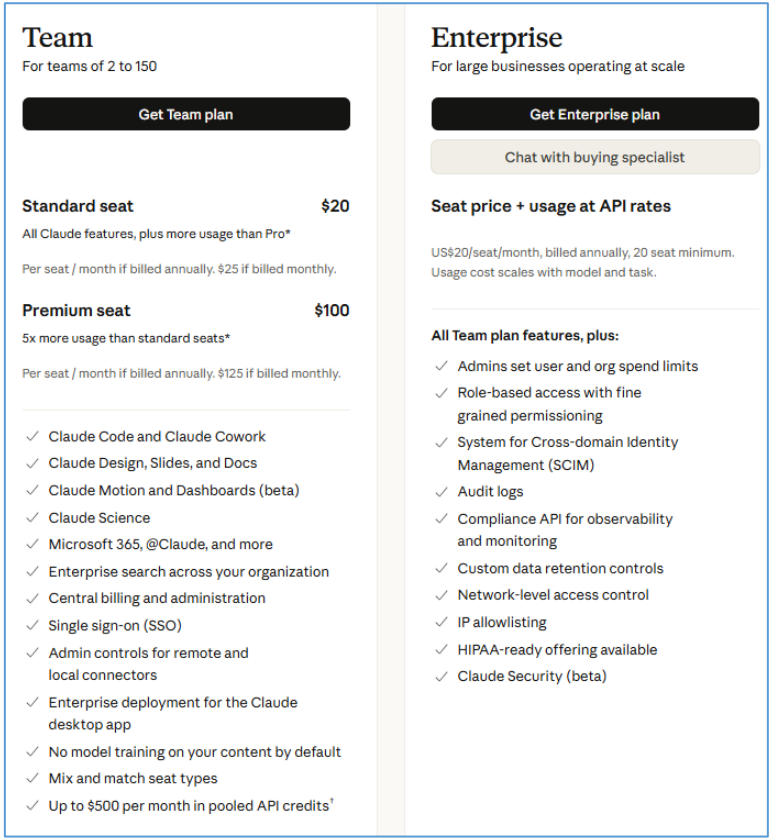
Per month

Try Claude

Everything in Pro, plus:

- ✓ Choose 5x or 20x more usage than Pro*
- ✓ Higher output limits for all tasks
- ✓ Early access to advanced Claude features
- ✓ Priority access at high traffic times
- ✓ $100 to $200/month in API credits to build on the Claude Platform†

Claude pricing for Team & Enterprise



Team

For teams of 2 to 150

Get Team plan

Standard seat **$20**

All Claude features, plus more usage than Pro*

Per seat / month if billed annually. $25 if billed monthly.

Premium seat **$100**

5x more usage than standard seats*

Per seat / month if billed annually. $125 if billed monthly.

- ✓ Claude Code and Claude Cowork
- ✓ Claude Design, Slides, and Docs
- ✓ Claude Motion and Dashboards (beta)
- ✓ Claude Science
- ✓ Microsoft 365, @Claude, and more
- ✓ Enterprise search across your organization
- ✓ Central billing and administration
- ✓ Single sign-on (SSO)
- ✓ Admin controls for remote and local connectors
- ✓ Enterprise deployment for the Claude desktop app
- ✓ No model training on your content by default
- ✓ Mix and match seat types
- ✓ Up to $500 per month in pooled API credits†

Enterprise

For large businesses operating at scale

Get Enterprise plan

Chat with buying specialist

Seat price + usage at API rates

US$20/seat/month, billed annually, 20 seat minimum. Usage cost scales with model and task.

All Team plan features, plus:

- ✓ Admins set user and org spend limits
- ✓ Role-based access with fine grained permissioning
- ✓ System for Cross-domain Identity Management (SCIM)
- ✓ Audit logs
- ✓ Compliance API for observability and monitoring
- ✓ Custom data retention controls
- ✓ Network-level access control
- ✓ IP allowlisting
- ✓ HIPAA-ready offering available
- ✓ Claude Security (beta)

Claude Models

Fable 5.1

Next generation intelligence for long-running agents

Prompt caching	
Read	**$0.25 / MTok**
Write	**$12.50 / MTok**
Input	**$10 / MTok**
Output	**$50 / MTok**

Opus 5.5

Daily driver for agentic coding and enterprise work

Prompt caching	
Read	**$0.20 / MTok**
Write	**$5 / MTok**
Input	**$4 / MTok**
Output	**$20 / MTok**

Sonnet 5.5

High-performance model for coding and agents

Prompt caching	
Read	**$0.10 / MTok**
Write	**$2.50 / MTok**
Input	**$2 / MTok**
Output	**$10 / MTok**

Haiku 5.5

Fastest, most cost-efficient model

Prompt caching	
Prompts ≤ 100K tokens	
Read	**$0.01 / MTok**
Write	**$0.125 / MTok**
Prompts > 100K tokens	
Read	**$0.05 / MTok**
Write	**$0.625 / MTok**
Input	
Prompts ≤ 100K tokens	**$0.10 / MTok**
Prompts > 100K tokens	**$0.50 / MTok**
Output	
Prompts ≤ 100K tokens	**$0.50 / MTok**
Prompts > 100K tokens	**$2.50 / MTok**

Claude code installation

The claude code can be installed in 3 different ways:

1. Claude desktop installation
2. Claude code in terminal
3. Claude code as Visual Code Studio extension

Claude code installation

Claude desktop installation:

1. You can download the claude as desktop software using Download Claude | Claude by Anthropic
2. You install the executable file and open the claude desktop software.
3. Now, you will get access to most common claude products like claude web, claude code, claude cowork etc.
4. You can choose the claude code tab and start using claude code.

Claude code installation

Steps to install Claude code in terminal:

1. npm install -g @anthropic-ai/claude-code
2. claude –version
3. Make sure, you have **gemma4:cloud** cloud model linked from local machine (ollama list)
4. ollama launch claude
5. Run the prompt in terminal:
6. "Create index.html file in the current folder. It should have heading as Hello World with yellow background color. Finally save this file into the workspace.“
7. Open the created index.html. Congratulations!! Your Claude Code is installed successfully!!

Claude code installation

Steps to install Claude code as extension in VSCode:

1. Open the VS code.

   Click on ‘Extensions’ icon located on left hand side icon list.

2. Search for “Claude code” and click on Install button.

3. Restart the VSCode and make sure you get below claude code icon in VSCode.



Claude slash commands

- Claude slash commands are shortcuts that let you control Claude's behavior, manage conversations, work with files, execute development tasks, and customize your workflow.
- They are especially useful in Claude Code.
- A slash command begins with / and is normally entered at the beginning of your message in a Claude Code session.

Understanding slash command syntax

Syntax	Meaning	Example
`/command`	Command without arguments	`/help`
`/command <arg>`	Required argument	`/add-dir ../shared`
`/command [arg]`	Optional argument	`/resume my-session`
`/command [options]`	Optional flags or settings	`/code-review high --fix`
`/custom-name`	User-defined skill or plugin command	`/deploy-staging`

Types of slash commands

1. **Built-in commands:** Implemented by Claude Code itself. Examples: /help, /clear, /model, /permissions.
2. **Bundled skills:** Reusable task instructions that Claude executes. Examples: /simplify, /verify, /batch.
3. **Custom skills and commands:** Created by you or your team. Examples: /write-tests, /release-notes, /deploy.
4. **Plugin and MCP commands:** Exposed by installed extensions or connected MCP servers. Availability depends on your setup.

Slash commands categories

Essential commands and session management	Project initialization, files and memory
Planning and reasoning controls	Code review, debugging, refactoring and verification
Git, pull requests and parallel development	Usage, context, cost and session diagnostics
Settings, permissions, appearance and input controls	Models, accounts and authentication
MCP, plugins, skills, agents and hooks	Research, browser automation, design and content-generation commands
Account, platform, cloud and integration utilities	Bundled skills for specialized tasks

Slash commands - Essential commands and session management

Slash command	Meaning	Usage
/help	Displays help and available commands.	/help
/clear	Starts a fresh conversation with empty context. Your saved project instructions and previous session are not necessarily deleted. Aliases: /reset, /new	/clear
/compact [instructions]	Summarizes the existing conversation to free context space while preserving relevant information.	/compact preserve the API design, changed files
/resume [session]	Reopens a previous conversation by name or ID, or presents a session picker. Alias: /continue	/resume payment-refactor
/rename [name]	Renames the current session so it is easier to identify later.	/rename payment-api-refactor

Slash commands - Essential commands and session management

Slash command	Meaning	Usage
/branch [name]	Creates a separate conversational branch so you can explore another approach without losing the original conversation.	/branch try-event-driven-design
/rewind	Opens checkpoint options to roll back conversation and/or code changes, or summarize from an earlier point. Aliases: /checkpoint, /undo	/rewind
/export [filename]	Exports the conversation as plain text, either to a named file or through an interactive export option.	/export debugging-session.txt
/copy [N]	Copies the latest assistant response or a numbered earlier response to the clipboard. Code-containing responses may offer a selection menu.	/copy 2
/btw [question]	Asks a quick side question without adding that exchange to the main conversation history.	/btw why does this project use Redis?

Slash commands - Essential commands and session management

Slash command	Meaning	Usage
/recap	Produces a brief, on-demand summary of the current session.	/recap
/exit	Exits the CLI session. Alias: /quit	/exit

Slash commands - Project initialization, files and memory

Slash command	Meaning	Usage
/init	Inspects a project and generates a starter CLAUDE.md with relevant project guidance.	/init
/memory	Views or edits CLAUDE.md instructions and manages supported auto-memory features.	/memory
/add-dir <path>	Adds another working directory to the current session's accessible locations.	/add-dir ../shared
/cd <path>	Changes the session's working directory while retaining the conversation.	/cd ../backend

Slash commands - Project initialization, files and memory

Slash command	Meaning	Usage
/context [all]	Visualizes context usage and highlights large consumers. all expands the breakdown.	/context all
/autocompact [auto|tokens]	Configures when automatic context compaction occurs, on supported versions.	/autocompact auto
/import [source]	Imports configuration from tools like Codex, Gemini, Cursor, subject to version and provider restrictions.	/import codex --dry-run
/update-config [request]	Uses a bundled skill to make requested settings changes by editing configuration files.	/update-config enable a hook for test failures

Slash commands - Planning and reasoning controls

Slash command	Meaning	Usage
/plan [description]	Enters Plan Mode to design a solution before making changes.	/plan migrate the REST API to GraphQL
/model [model]	Shows model options or switches the active model.	/model
/effort [level]	Adjusts the reasoning effort level supported by the model.	/effort high
/advisor [model|off]	Enables or disables the optional second-model advisor, where supported.	/advisor sonnet
/fast [on|off]	Toggles fast mode, where available.	/fast on
/goal [condition|clear]	Sets or clears a goal for supported multi-turn autonomous work.	/goal all unit tests pass
/autofix-pr [prompt]	Starts a cloud session that watches a pull request and attempts fixes for CI failures or review comments.	/autofix-pr only fix lint and type errors

Slash commands - Code review, debugging, refactoring and verification

Slash command	Meaning	Usage
/code-review [options] [target]	Reviews code changes for correctness bugs and other issues. You can target the current diff, a branch, path or pull request. The available review effort and flags vary by version.	/code-review high --fix
/security-review	Examines branch changes for security problems such as injection vulnerabilities, authentication mistakes and data exposure.	/security-review
/simplify [target]	Runs a bundled skill that reviews changed code for duplication, unnecessary complexity, efficiency and abstraction problems, then applies improvements.	/simplify src/services/payment.ts
/verify	Runs the bundled verification workflow to check that a change actually works. Depending on the project, this may involve building and running the app and observing its behavior.	/verify

Slash commands - Code review, debugging, refactoring and verification

Slash command	Meaning	Usage
/run	Launches and drives your project application so Claude can inspect the result, where the relevant project-specific setup is available.	/run
/debug [description]	Loads a debugging skill that enables or examines session debug logs and helps diagnose runtime issues.	/debug the server exits after the first request
/doctor [prompt-audit [path]]	Checks the Claude Code installation, settings, extensions and instruction files, then proposes fixes. The prompt-audit option checks instructions on supported versions. Alias: /checkup	/doctor prompt-audit

Slash commands - Git, pull requests and parallel development

Slash command	Meaning	Usage
/diff	Opens an interactive view of uncommitted changes and changes made during Claude's turns.	/diff
/code-review [options] [target]	Reviews a local diff or a specified PR, branch or path.	/code-review high 1234
/batch <instruction>	Breaks a large codebase-wide change into parallel work units and coordinates implementation.	/batch migrate src/ from JavaScript to TypeScript
/background [prompt]	Detaches the current session to run as a background agent.	/background finish the tests and summarize failures
/fork [prompt]	Copies the conversation into a separate background session.	/fork investigate a different caching strategy

Slash commands - Git, pull requests and parallel development

Slash command	Meaning	Usage
/subtask <task>	Starts a background subagent that returns its findings to the current conversation.	/subtask inspect database query performance
/tasks	Lists and manages background tasks, including completed tasks.	/tasks
/list-agents	Lists available subagents and other sessions in setups that support cross-session messaging.	/list-agents
/stop	Stops the background session to which you are attached or sending a command.	/stop
/teleport	Brings a cloud session into your terminal.	/teleport
/remote-control	Makes a local session accessible through supported remote-control clients.	/remote-control
/web-setup	Connects GitHub account for supported cloud sessions.	/web-setup

Slash commands - Usage, context, cost and session diagnostics

Slash command	Meaning	Usage
/usage	Session cost, plan usage limits and activity statistics, depending on account.	/usage
/context	Visualizes current context-window consumption.	/context all
/status	Displays version, model, account and connectivity information.	/status
/statusline	Configures the CLI status line.	/statusline
/insights	Generates a report about recent sessions and usage patterns.	/insights
/release-notes	Displays release notes for available versions.	/release-notes
/feedback [report]	Opens a product feedback workflow.	/feedback the terminal display flickers

Slash commands - Usage, context, cost and session diagnostics

Slash command	Meaning	Usage
/bug [report]	Reports a bug with a consent step for sharing session information.	/bug the command fails on startup
/heapdump	Creates diagnostic memory snapshots for investigating high memory usage. Hidden from the normal command menu on documented versions.	/heapdump
/recap	Summarizes the current session in one line.	/recap

Slash commands - Settings, permissions, appearance and input controls

Slash command	Meaning	Usage
/config [key=value ...]	Opens settings or sets supported preferences directly.	/config theme=dark
/permissions	Manages tool allow, ask and deny rules.	/permissions
/sandbox	Configures or toggles sandbox mode on supported platforms.	/sandbox
/theme	Changes terminal appearance, including supported light, dark and accessibility themes.	/theme
/color [color|default]	Changes the prompt-bar color for the current session.	/color purple
/keybindings	Opens or creates the keyboard shortcut configuration file.	/keybindings
/terminal-setup	Helps configure terminal-specific keyboard and clipboard behavior.	/terminal-setup

Slash commands - Settings, permissions, appearance and input controls

Slash command	Meaning	Usage
/tui [default|fullscreen]	Selects the terminal UI renderer where supported.	/tui fullscreen
/focus [on|off]	Toggles a reduced display focused on prompts and final results in supported environments.	/focus
/scroll-speed	Adjusts mouse-wheel scroll speed in supported full-screen terminals.	/scroll-speed
/output-style [style]	Lists or switches output styles on supported versions.	/output-style concise
/effort [level]	Adjusts reasoning effort; it is also a model-control command.	/effort medium
/fast [on|off]	Controls fast mode when supported.	/fast off
/voice [hold|tap|off]	Configures voice dictation where available.	/voice hold

Slash commands - Models, accounts and authentication

Slash command	Meaning	Usage
/login	Signs in to your Anthropic account.	/login
/logout	Signs out of the current Anthropic account.	/logout
/model [model]	Selects a model and, where supported, saves the default for new sessions.	/model
/advisor [model|off]	Configures the optional second-model advisor on supported versions and accounts.	/advisor off
/upgrade	Opens the account upgrade flow when available.	/upgrade
/rate-limit-options	Displays options for continuing when you hit usage limit.	/rate-limit-options
/usage-credits	Opens usage-credit configuration or a request-to-admin workflow where supported.	/usage-credits
/passes	Offers eligible users to share a free week of Claude Code.	/passes
/privacy-settings	Opens privacy settings on supported plans.	/privacy-settings

Slash commands - MCP, plugins, skills, agents and hooks

Slash command	Meaning	Usage
/mcp	Manages MCP server connections and authentication.	/mcp
/plugin [subcommand]	Lists, installs, enables or disables plugins.	/plugin list
/reload-plugins [--force]	Reloads active plugins without restarting the session.	/reload-plugins
/reload-skills	Re-scans skill and command directories for changes.	/reload-skills
/skills	Lists available skills and manages their visibility where supported.	/skills
/skill-doctor	Analyzes skill usage and context cost on supported versions.	/skill-doctor

Slash commands - MCP, plugins, skills, agents and hooks

Slash command	Meaning	Usage
/agents	Helps you manage or create specialized subagents.	/agents
/hooks	Shows configured event hooks.	/hooks
/ide	Manages IDE integrations and displays their status.	/ide
/list-agents	Lists agents and sessions in supported cross-session setups.	/list-agents
/plugin-authoring	Loads guidance for writing plugins or extensions where available.	/plugin-authoring

Slash commands - Research, browser automation, design and content-generation commands

Slash command	Meaning	Usage
/deep-research <question>	Runs a research workflow that searches, cross-checks sources and synthesizes a cited report.	/deep-research Compare PostgreSQL and CockroachDB for a multi-region SaaS
/claude-in-chrome [task]	Uses the Chrome integration to carry out browser tasks, where enabled.	/claude-in-chrome inspect the checkout page for console errors
/chrome	Configures Claude in Chrome integration.	/chrome
/dataviz [request]	Loads guidance for designing charts, graphs and dashboards.	/dataviz create a monthly revenue dashboard

Slash commands - Research, browser automation, design and content-generation commands

Slash command	Meaning	Usage
/design [brief]	Creates design artboards, mockups or screen flows through the supported Claude Design integration.	/design create a settings screen for a mobile banking app
/design-login	Authorizes access to the design system for supported design-sync workflows.	/design-login
/design-sync [hint]	Synchronizes a React design system for use with Claude Design, where available.	/design-sync Acme design system
/slides [brief]	Generates a presentation through the supported Claude Slides integration.	/slides quarterly engineering review
/artifact-capabilities	Loads guidance about capabilities supported by published artifacts.	/artifact-capabilities

Slash commands - Research, browser automation, design and content-generation commands

Slash command	Meaning	Usage
/artifact-diagramming	Loads guidance for creating diagrams in artifacts.	/artifact-diagramming
/artifacts	Lists and opens your artifacts or artifacts shared with you, where supported.	/artifacts
/claude-api [subcommand]	Loads Claude API and Managed Agents reference material; supports documented API-related subcommands.	/claude-api migrate
/workflow-authoring	Loads guidance for authoring dynamic workflows.	/workflow-authoring
/workflows	Opens the workflow progress view for supported dynamic workflows.	/workflows
/run-skill-generator	Helps configure a project-specific skill so /run and /verify can operate on the application.	/run-skill-generator

Slash commands - Account, platform, cloud and integration utilities

Slash command	Meaning	Usage
/desktop	Continues the session in the Claude Code desktop application.	/desktop
/mobile	Displays a QR code for the Claude mobile app.	/mobile
/remote-env	Selects a default cloud environment for supported sessions.	/remote-env
/schedule [description]	Creates or manages cloud routines for scheduled or event-driven execution.	/schedule run a weekday summary of open PRs
/background [prompt]	Detaches the current session as a background agent.	/background finish the migration and run tests
/teleport	Pulls a cloud session into the local terminal.	/teleport

Slash commands - Account, platform, cloud and integration utilities

Slash command	Meaning	Usage
/remote-control	Enables remote access to the current session.	/remote-control
/install-github-app	Sets up the Claude GitHub app and supported repository integration.	/install-github-app
/install-slack-app	Starts the Slack app installation and authorization flow.	/install-slack-app
/setup-bedrock	Configures Amazon Bedrock authentication and model settings.	/setup-bedrock
/setup-vertex	Configures Google Cloud's Agent Platform authentication and model settings.	/setup-vertex
/powerup	Presents interactive lessons about Claude Code features.	/powerup
/radio	Opens Claude FM, where supported.	/radio
/stickers	Opens the Claude Code stickers ordering workflow.	/stickers

Slash commands - Bundled skills for specialized tasks

Slash command	Meaning	Usage
/batch	Coordinates large changes across independent units of a codebase.	/batch convert the legacy API to TypeScript
/simplify	Reviews changed code for duplication, complexity and efficiency.	/simplify focus on duplicated validation logic
/verify	Helps verify that the application works after changes.	/verify
/run	Launches and exercises the application when configured.	/run
/debug	Helps investigate runtime problems and debug logs.	/debug investigate intermittent API timeouts
/doctor	Diagnoses configuration, setup and instruction problems.	/doctor
/deep-research	Produces a source-backed research report.	/deep-research evaluate message brokers for event-driven systems

Slash commands - Bundled skills for specialized tasks

Slash command	Meaning	Usage
/dataviz	Provides guidance for charts and dashboards.	/dataviz create an accessible sales dashboard
/claude-api	Loads Claude API and Managed Agents documentation.	/claude-api migrate
/fewer-permission-prompts	Analyzes recurring read-only tool calls and suggests a scoped allowlist.	/fewer-permission-prompts
/update-config	Helps edit configuration settings in response to a request.	/update-config add a Stop hook that runs lint
/plugin-authoring	Loads extension-authoring guidance.	/plugin-authoring
/workflow-authoring	Loads dynamic-workflow authoring guidance.	/workflow-authoring

Slash commands - Bundled skills for specialized tasks

Slash command	Meaning	Usage
/run-skill-generator	Helps configure application verification skills.	/run-skill-generator
/artifact-capabilities	Explains supported artifact capabilities.	/artifact-capabilities
/artifact-diagramming	Provides diagramming guidance for artifacts.	/artifact-diagramming

Steps to create custom slash command

1. Create “.claude” directory inside your project folder.
2. Create “commands” directory inside “.claude” folder.
3. Create .md file inside “commands” folder where name of the .md file is same as custom slash command. Suppose we wish to create /news slash command that gives recent news then you will create news.md file.
4. Copy below contents inside news.md file.

   ---

   description: Prints today's Indian news

   allowed-tools: TavilySearchResults

   Invoke the TavilySearchResults tool with the prompt 'What are the today's headlines in India?’
5. Now exit from current session using /exit
6. Start new claude session. Now you can see the custom slash command /news.
7. Run /news command, it will show you today’s headlines.

How Claude code works?

- Claude code is an "agentic coding tool". This tool interacts with your project, runs commands, integrate with your development tools.
- Claude code uses "Agentic Loop“ which has 3 phases ‘Gather context’, ‘Take action’ and ‘Verify results’. This loop continues until we get satisfied results.



Claude.md file

- The CLAUDE.md file is an instruction and context file used by Claude Code to understand how you want it to work within a software project.
- Think of it as a project's handbook for Claude.
- Instead of repeating your coding standards, architecture decisions, and development instructions in every prompt, you write them once in CLAUDE.md, and Claude can use them when working on the project.

Why is CLAUDE.md important?

Project context: Explains what the application does, its architecture, and its technology stack.

Coding standards: Defines naming conventions, formatting rules, and preferred coding patterns.

Development instructions: Specifies how to build, test, and run the application.

Consistency: Helps Claude follow the same rules across different files and tasks.

Reduced repetition: Avoids having to include the same instructions in every conversation.

For example, if you're working on a React application, you can tell Claude to use TypeScript, follow a particular folder structure, and run tests before finishing a change.

Sample CLAUDE.md for web application

Project Overview

This is a React application built with TypeScript.

Tech Stack

- Frontend: React, TypeScript, Tailwind CSS
- Backend: Node.js and Express
- Database: PostgreSQL

Coding Guidelines

- Use TypeScript for all new frontend code.
- Prefer functional components and React hooks.
- Follow the existing project structure.
- Reuse existing components instead of duplicating code.

Sample CLAUDE.md for web application

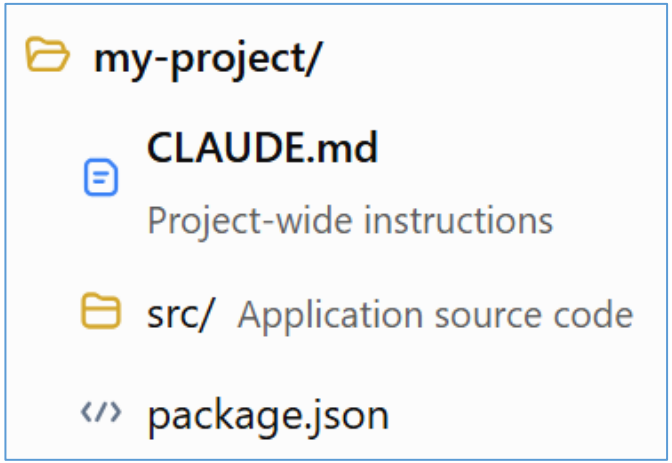
Development Commands

- Install dependencies: npm install
- Start development: npm run dev
- Run tests: npm test
- Build: npm run build

Important Rules

- Do not introduce new dependencies without approval.
- Do not hardcode API keys or secrets.
- Add tests for new functionality.
- Run relevant tests after making changes.

Where should you create CLAUDE.md?



- Place CLAUDE.md in the project root for general instructions.
- You can also create additional CLAUDE.md files in subdirectories for more specific guidance.
- To get started, open Claude Code in your project directory and run: /init
- Claude Code will generate a starting CLAUDE.md based on the project. Review the result and correct anything inaccurate.

CLAUDE.md locations

You can find CLAUDE.md in below possible locations:

1. Root directory of your project
2. Module directory of your project
3. User’s project directory, for example c:\Users\anand

Limitations of CLAUDE.md

CLAUDE.md is not a configuration file that strictly enforces every rule. It provides instructions that Claude attempts to follow.

- It does not guarantee error-free code.
- It does not automatically grant permission to execute restricted commands.
- It does not replace tests, security checks, or code review.
- Long or contradictory instructions can reduce their effectiveness.

Settings.json file

The settings.json in Claude Code is the configuration file that controls how Claude Code behaves in your terminal.

Think of it as Claude Code's control panel. Instead of changing settings every time you start Claude Code, you can configure them once in this file.

Where settings.json is located?

File location	Purpose
`~/.claude/settings.json`	Global settings for all projects
`.claude/settings.json`	Settings for the current project (can be shared with your team)
`.claude/settings.local.json`	Personal settings for the current project

Settings.json file

What can you configure in settings.json?

1. Permission modes: Whether Claude asks before editing files or running commands.
2. Allowed commands: Commands Claude can execute without asking.
3. Denied commands: Commands Claude cannot execute.
4. Model: Your preferred Claude model.
5. Environment variables: Variables available to Claude Code.
6. Hooks: Commands or scripts triggered by certain events.
7. Plugins and other preferences.

Settings.json file

Sample settings.json

```
{
  "permissions": {
    "defaultMode": "acceptEdits",
    "allow": [
      "Bash(npm run *)",
      "Bash(git status)"
    ],
    "deny": [
      "Read(./.env)"
    ]
  }
}
```

settings.json vs CLAUDE.md

Feature	settings.json	CLAUDE.md
Configure permission modes	Yes	No
Allow/deny commands	Yes	No
Configure hooks	Yes	No
Give Claude project instructions	Not its purpose	Yes
Define coding standards	Not its purpose	Yes
Explain project architecture	Not its purpose	Yes

Claude Modes

- Claude Code has different permission modes that control how much freedom Claude has to read files, modify code, execute terminal commands, and perform actions without asking for your approval. You can change the mode using command “claude --permission-mode acceptEdits”

Mode	What Claude can do	Approval required?
Manual (`default`)	Read files and analyze code	Yes, before edits and commands
Accept Edits (`acceptEdits`)	Read and modify files	Yes, before other restricted actions
Plan (`plan`)	Explore code and create implementation plans	Cannot modify source files
Auto (`auto`)	Execute tasks with background safety checks	Automatically approves safe actions
Don't Ask (`dontAsk`)	Execute previously approved tools	Unapproved actions are denied
Bypass Permissions (`bypassPermissions`)	Execute actions without normal permission prompts	No normal permission prompts

Thank you!!